

distbatch-infer: CUDA·OpenMP·gRPC 기반 분산 배치 추론 엔진

Parallel & Distributed Computing (CSEG414/5414) — Course Project Final Report

대상 환경: NVIDIA GeForce RTX 5060 Ti (Blackwell, sm_120, cc 12.0, 36 SM) · CUDA 12.9 · gRPC 1.30.2 / Protobuf 3.12.4 · GCC 12.3 · CMake 3.29 · OpenMP 4.5 · Linux

1. Abstract & Problem Definition

1.1 대상 애플리케이션

본 프로젝트는 Transformer 블록 forward 연산(self-attention + FFN)을 직접 작성한 CUDA 커널로 구현하고, 이를 gRPC 워커 풀로 서빙하는 분산 배치 추론 엔진이다. Transformer 블록은 거대 언어 모델(LLM) 추론의 핵심 연산 단위이며, 실제 추론 서비스는 다수의 클라이언트 요청을 받아 배치로 묶어 GPU에서 가속하고 결과를 되돌려주는 다단계 파이프라인 구조를 가진다. 본 시스템은 이 구조를 단일 머신(localhost, 다중 프로세스/포트)에서 재현한다.

입력은 (B, S, D) 텐서(배치 × 시퀀스 길이 × hidden dim)이며, 다음 순서로 forward를 수행한다:

```
QKV projection → head split → scores=QKT → softmax(scale) → context=scores·V  
→ head merge → output projection → FFN up → GELU → FFN down
```

단순화를 위해 LayerNorm/residual/dropout은 제외했다(향후 과제). 데이터 타입은 FP32로 고정한다.

1.2 계산 병목 (Computational Bottlenecks)

Transformer 블록의 본질적 병목은 다음과 같으며, 본 보고서의 실험에서 정량적으로 확인한다.

- GEMM (compute-bound):** QKV/output/FFN projection은 대형 행렬곱으로 연산량이 지배적이다. roofline 상 compute roof 근처에 위치한다.
- Attention의 $O(S^2)$ 비용:** QK^T 와 $scores \cdot V$ 는 시퀀스 길이 S에 대해 제곱으로 증가한다.
- 메모리 바운드 elementwise:** softmax, GELU는 산술 강도(arithmetic intensity)가 낮아 DRAM 대역폭에 묶인다.
- 분산 계층 오버헤드:** 직렬화(protobuf), 네트워크 전송, H2D/D2H 복사, 배치 큐 대기 — 페이로드가 클수록 비중이 커진다.

1.3 프레임워크 선정 이유

요구	선택	근거
병렬 가속 (HPC)	CUDA	행렬곱·attention을 GPU에서 직접 작성한 커널로 가속. cuBLAS/cuDNN 등 라이브러리 연산은 사용하지하고 전부 hand-written <code>__global__</code> 커널로 구현하여 병렬 프로그래밍 역량을 직접 입증.
병렬 가속 (CPU)	OpenMP	dispatcher에서 여러 요청 입력을 하나의 배치 버퍼로 모으고(gather) 응답을 되돌려 나누는(scatter) 대용량 float 배열 복사를 <code>#pragma omp parallel for</code> 로 병렬화.
분산 미들웨어	gRPC + Protobuf	다단계 파이프라인(Client→Dispatcher→Worker)을 명확한 RPC 경계로 분리. protobuf로 텐서 직렬화.

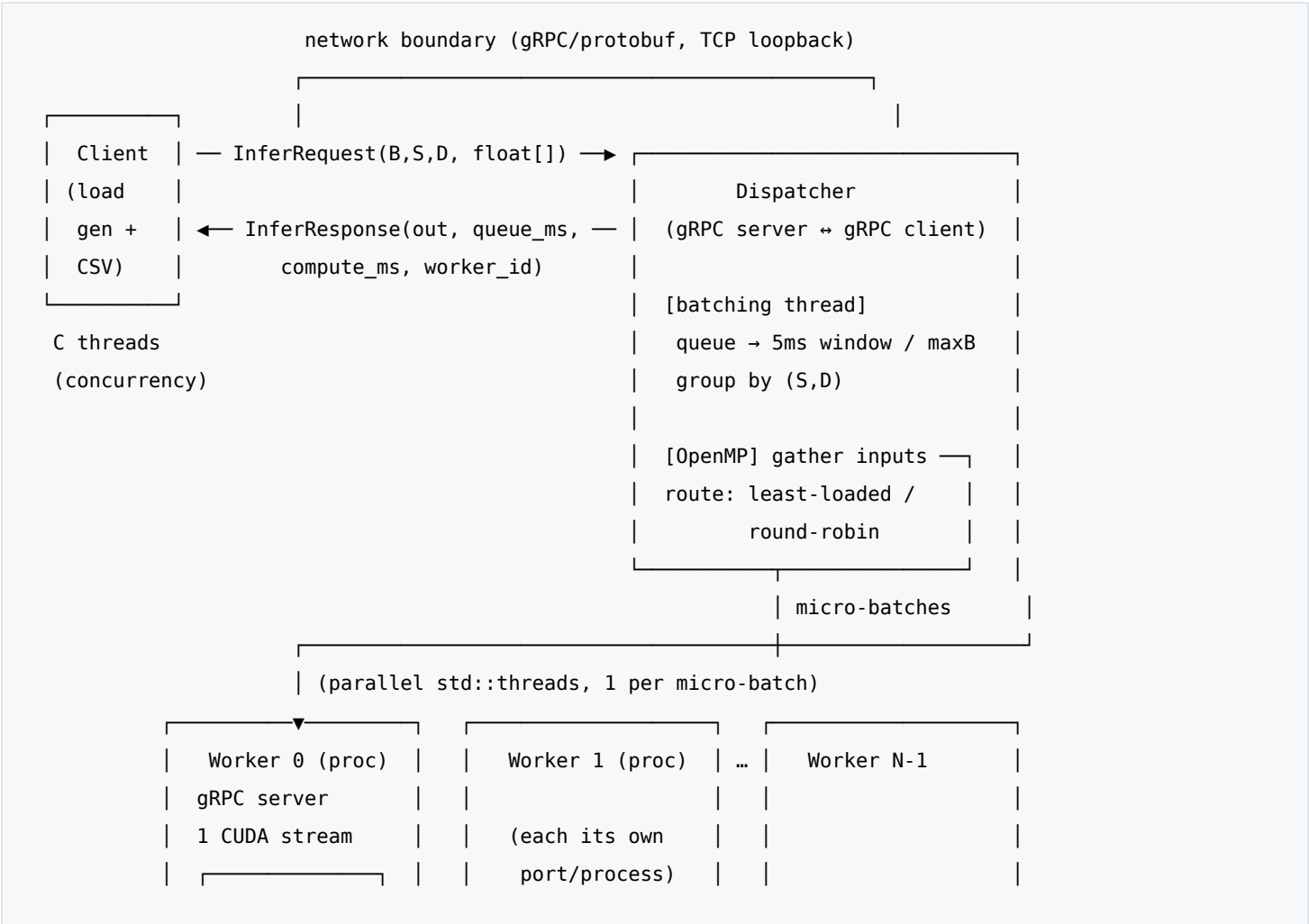
이 조합은 대학원 요건을 동시에 충족한다: (1) 복수 HPC 프레임워크(CUDA+OpenMP), (2) 다단계 분산 파이프라인 (Client→Dispatcher→Worker), (3) 이중 가속(CPU 전처리/배치 + GPU 연산).

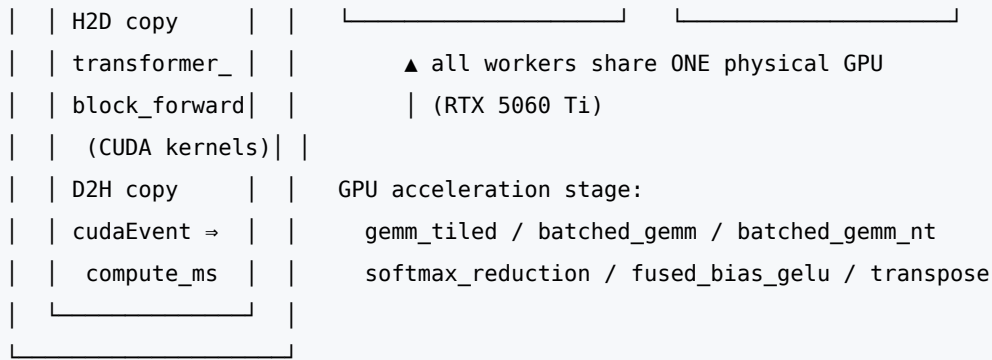
1.4 설계 근거 (Design Rationale)

주요 설계 결정과 그 이유는 다음과 같다.

- **3단계 분리(Client / Dispatcher / Worker):** 부하 생성, 배치·라우팅, GPU 연산을 독립 프로세스로 분리해 구조적 디커플링을 달성. Worker를 포트별 독립 프로세스로 두어 수평 확장(워커 수)을 단순 설정으로 가능케 함.
- **Dispatcher에서의 동적 배치:** 추론은 단건보다 배치 처리 시 GPU 효율이 급격히 상승(§ 4.1)하므로, 짧은 윈도우(5ms)로 요청을 모아 micro-batch를 구성. 지연-처리량 trade-off를 윈도우/최대배치 파라미터로 조절.
- **직접 작성 CUDA 커널(라이브러리 금지):** 과제의 병렬 가속 역량 평가 취지에 맞춰 GEMM/softmax/GELU/transpose를 전부 hand-written __global__ 커널로 구현. tiling·warp reduction·fused elementwise 등 핵심 GPU 최적화 기법을 직접 적용.
- **Worker 단일 stream + mutex:** GPU 자원 경합을 단순화하고 정확한 cudaEvent 시간 측정을 보장. 다중 워커는 프로세스 수준 동시성으로 확보.
- **least-loaded 라우팅 + OpenMP gather/scatter:** in-flight 기반 부하 분산으로 워커 활용을 균등화하고, 대용량 텐서 복사를 CPU 멀티스레드로 가속해 dispatcher가 병목이 되지 않도록 함.
- **protobuf repeated float 평탄화:** 텐서를 단순 1D 배열로 직렬화해 구현을 단순화하되, 페이로드 바이트를 로깅해 직렬화/전송 병목을 정량 분석(§ 4.6).

2. System Architecture Diagram





- **Data flow:** Client가 평탄화된 (B·S·D) float 배열을 protobuf로 직렬화 → Dispatcher가 동일 shape 요청을 배치로 병합 → Worker가 GPU forward 후 결과 반환 → Dispatcher가 per-request로 분배 → Client가 latency/throughput 집계.
- **Network boundaries:** Client↔Dispatcher, Dispatcher↔Worker 두 개의 gRPC 경계. 모두 TCP(localhost). 메시지 상한 256MB.
- **Middleware stages:** Dispatcher가 클라이언트엔 InferService 서버, 워커엔 클라이언트로 동작하는 **2-hop 파이프라인**.
- **Thread/process organization:** Worker = 독립 프로세스(포트별). Dispatcher = 단일 프로세스 내 [gRPC 서버 스레드풀 + 전용 배칭 스레드 + micro-batch당 dispatch 스레드 + OpenMP 팀]. Client = C개 부하 스레드.
- **Parallel acceleration stages:** GPU(커널 6종) + CPU(OpenMP gather/scatter).

3. Implementation Details

3.1 직접 작성한 CUDA 커널 (제약: 라이브러리 GEMM/softmax/attention 사용 금지)

커널	역할	핵심 기법	thread/block
gemm_tiled	$C=A \cdot B$ (projection)	shared-memory 타일링, TILE=32, +1 padding으로 bank conflict 회피, 경계 가드	block 32×32 , grid $[N/32] \times [M/32]$
batched_gemm	scores · V	blockIdx.z=배치, 포인터 오프셋	$32 \times 32 \times \text{batch}$
batched_gemm_nt	$Q \cdot K^T$	K^T 를 materialize하지 않고 B 타일을 transpose 로드	$32 \times 32 \times \text{batch}$
softmax_reduction	row-wise softmax	warp당 1행, __shfl_xor_sync butterfly reduction(max/sum), scale · max-빼기로 수치안정	128 thread(4 warp)/block
fused_bias_gelu	GELU(x+bias)	bias-add와 tanh 근사 GELU를 단일 fused elementwise로, grid-stride	256 thread/block
split_heads/merge_heads	head 재배포	인덱스 디코딩 기반 transpose, 1 thread/element	256 thread/block
add_bias	broadcast bias	grid-stride elementwise	256 thread/block

transformer_block_forward는 위 커널을 묶어 § 1.1 데이터플로를 구성하며, 모든 중간 버퍼는 단일 cudaMalloc 영역을 block_scratch_partition으로 슬라이스해 사용한다.

3.2 미들웨어 구성 & 직렬화

- **gRPC**: `InsecureServerCredentials(localhost), SetMax{Receive,Send}MessageSize(256MB)` — 큰 텐서 페이지로 드 대응.
- **Protobuf 직렬화**: `InferRequest{batch_size, seq_len, hidden_dim, repeated float input, request_id}`, `InferResponse{repeated float output, compute_ms, queue_ms, request_id, worker_id}`. 텐서는 row-major 평탄화 `repeated float`. 직렬화 비용 분석을 위해 요청/응답 바이트 크기를 로깅한다(예: B=8,S=256,D=128 → 1 MB).
- 구버전 apt gRPC(1.30) 특성상 `RepeatedField`에 `iterator Assign`이 없어 `Resize+memcpy`로 텐서를 채운다(zero-copy에 가까운 경로).

3.3 배치 크기 & 배칭 전략 (Dispatcher)

- 요청을 큐에 모아 **배칭 윈도우(기본 5ms)** 또는 **max micro-batch** 도달 시 flush.
- 동일 (`seq_len`, `hidden_dim`) 요청을 배치 축으로 병합(다른 shape는 별도 그룹). 워커는 임의의 총 배치 크기를 런타임에 수용(scratch lazy 재할당).
- 실험은 배치 1–2048, 시퀀스 64–8192 범위로 스윕.

3.4 스레드/블록 구조 & 병렬화 전략

- **GPU**: § 3.1 표의 block 구성. GEMM은 $32 \times 32 = 1024$ thread/block(최대), elementwise는 256, softmax는 warp 단위.
- **OpenMP (CPU)**: dispatcher의 `send_microbatch`에서
- `gather`: `#pragma omp parallel for`로 각 요청 input을 배치 버퍼의 오프셋으로 memcpy.
- `scatter`: 워커 출력을 per-request 결과로 memcpy. 이 두 대용량 복사가 OpenMP의 측정 가능한 역할이다.
- **micro-batch 병렬 dispatch**: 한 배치 내 여러 micro-batch를 `std::thread`로 동시에 워커에 전송 → 다중 워커 동시 가동.

3.5 동기화 방법

- **Worker**: 단일 CUDA stream + scratch 공유 → `std::mutex`로 GPU 작업 직렬화. `cudaEvent`로 forward 구간만 측정해 `compute_ms` 반환.
- **Dispatcher**: 요청 큐는 `mutex+condition_variable`(배칭 윈도우는 `wait_for`). 라우팅은 `route_mu_` 보호 하 in-flight 카운터로 least-loaded 선택(또는 round-robin). 각 요청 도착시각 기록 → dispatch 직전까지를 `queue_ms`로 측정.
- **Client**: 부하 스레드는 `std::atomic` 인덱스로 작업 분배.

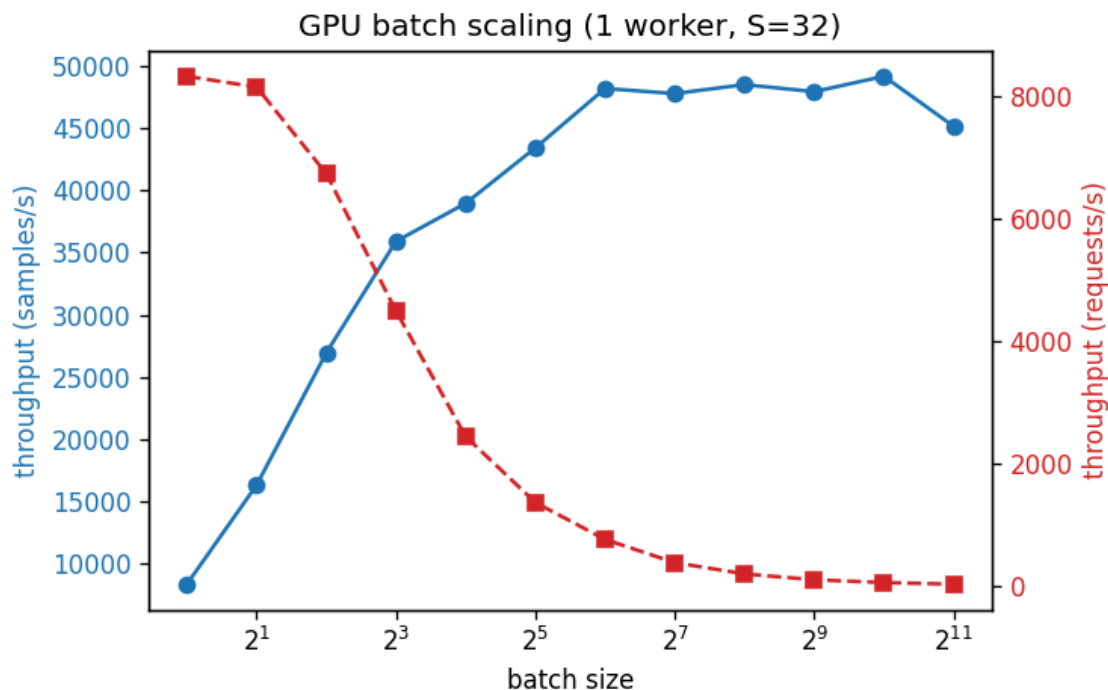
3.6 정확성 검증 (correctness)

- **커널 단위** (`tests/test_kernels.cu`): GPU FP32 vs double CPU 레퍼런스, `atol=1e-3`. 전 케이스 통과(최악 768^3 GEMM $5.3e-5$). transpose는 오차 0.
- **블록 단위** (`tests/test_block.cu` ↔ `tests/ref_block.py`): PyTorch 명시적 `x@w(row-major, nn.Linear 아님)` 레퍼런스와 단계별 비교, `atol=rtol=1e-2`. 최종 출력 `max_abs=3.8e-3`로 통과. gRPC 경로(`client --fixtures`)의 출력도 동일값으로 일치 → 서빙 경로의 수치 증가성 확인.

4. Performance Evaluation & Bottleneck Analysis

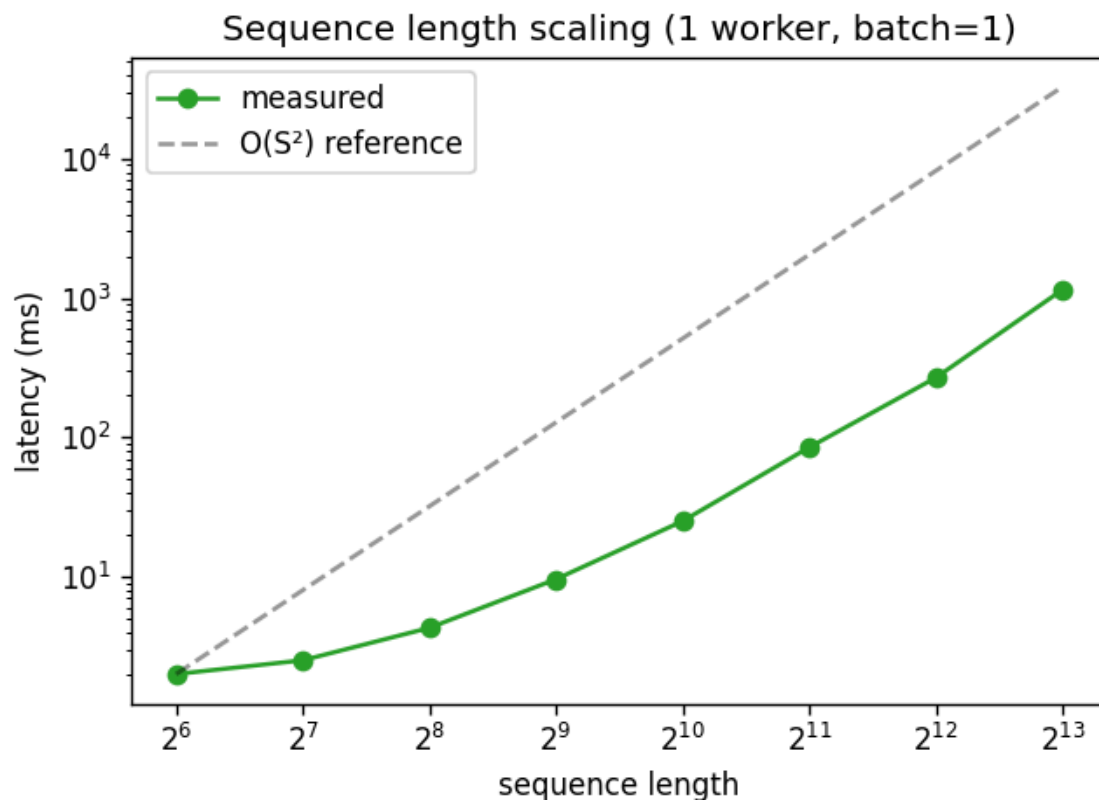
모든 그래프는 `scripts/run_experiments.sh`가 생성한 `results/*.png`이며, 데이터는 `results/*.csv`에 있다.

4.1 GPU batch size scaling



배치 1→2048(S=32, 1 worker). **samples/s가 8.3k에서 b=256-1024 구간 ~48-49k로 포화**하고 b=2048에서 45k로 소폭 하락. requests/s는 요청당 작업량 증가로 감소. → GPU가 충분히 채워지는 포화점을 정량화.

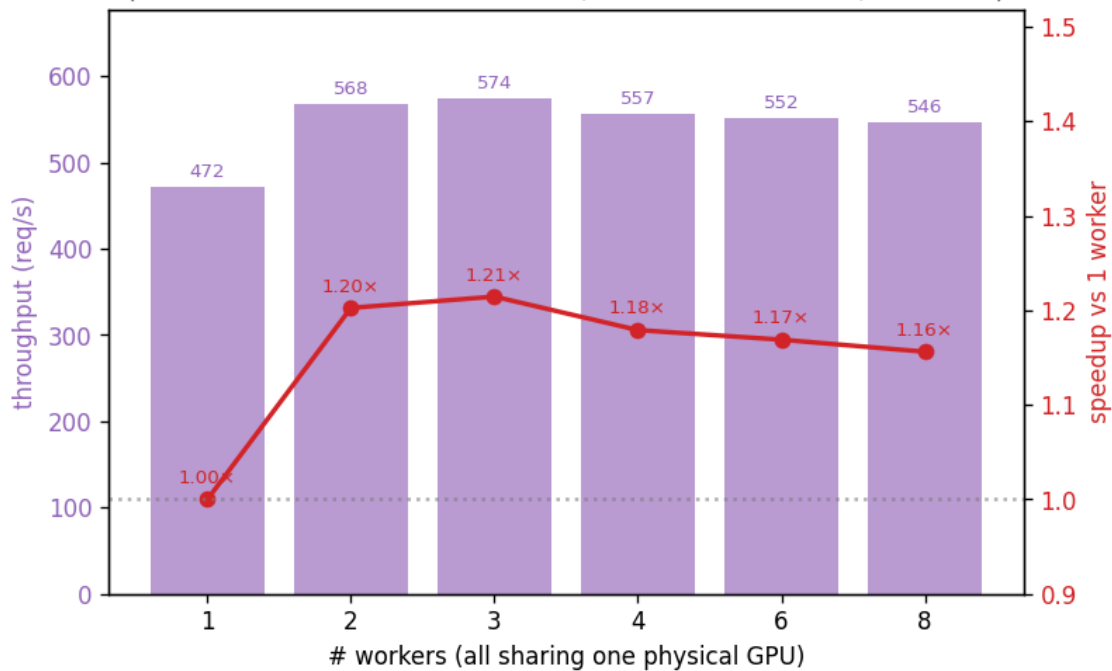
4.2 Sequence length scaling



시퀀스 64→8192(batch=1), log-log. mean latency 2.0ms(64)→25(1024)→268(4096)→**1143ms(8192)**. 작은 S는 오버 헤드 바운드로 평탄, 큰 S에서 기울기가 **O(S²) 기준선**에 수렴 → attention의 제곱 비용이 지배.

4.3 Worker/process scaling

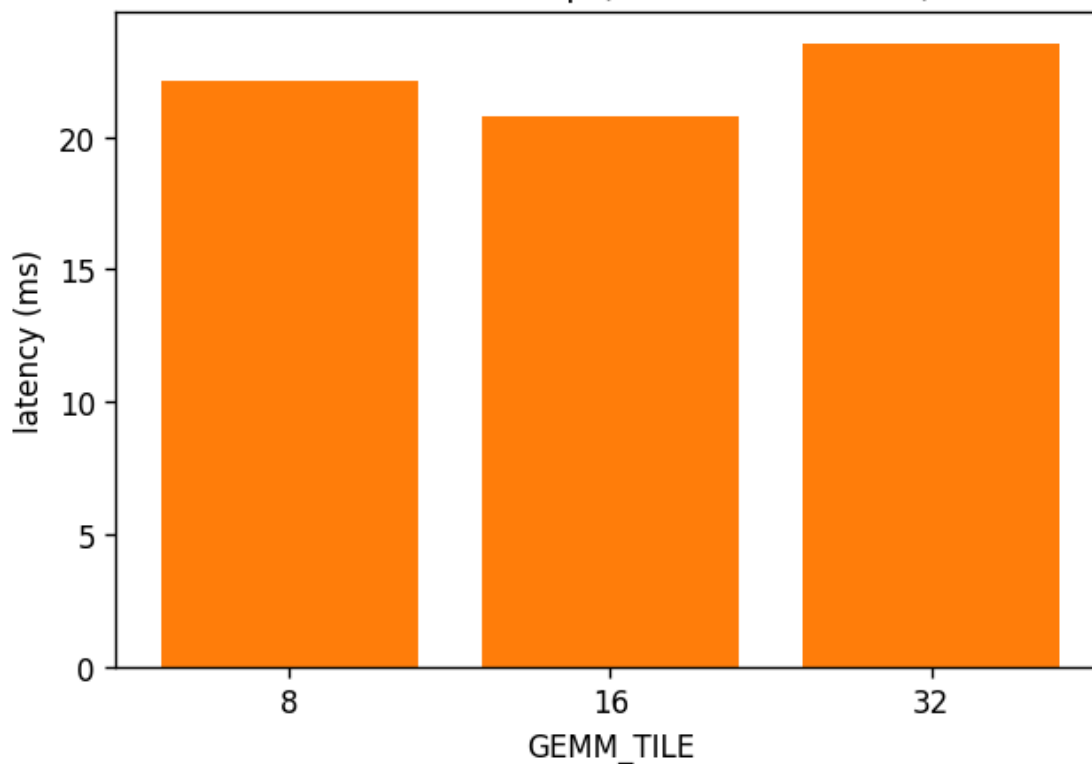
Worker scaling — throughput peaks at 2-3 workers, then declines
(all workers time-share one GPU; ideal would be $N\times$, off-chart)



워커 1→8(dispatcher, batch=8 S=256). throughput 472/568/574/557/552/546 req/s
($\approx 1.00/1.20/1.21/1.18/1.17/1.16\times$). 2-3 워커에서 정점 후 하락. 모든 워커가 단일 물리 GPU를 공유하므로
H2D/compute/D2H 오버랩으로 인한 소폭 이득 후, 추가 프로세스는 GPU 시간 공유 경합만 유발(아래 § 4.6).

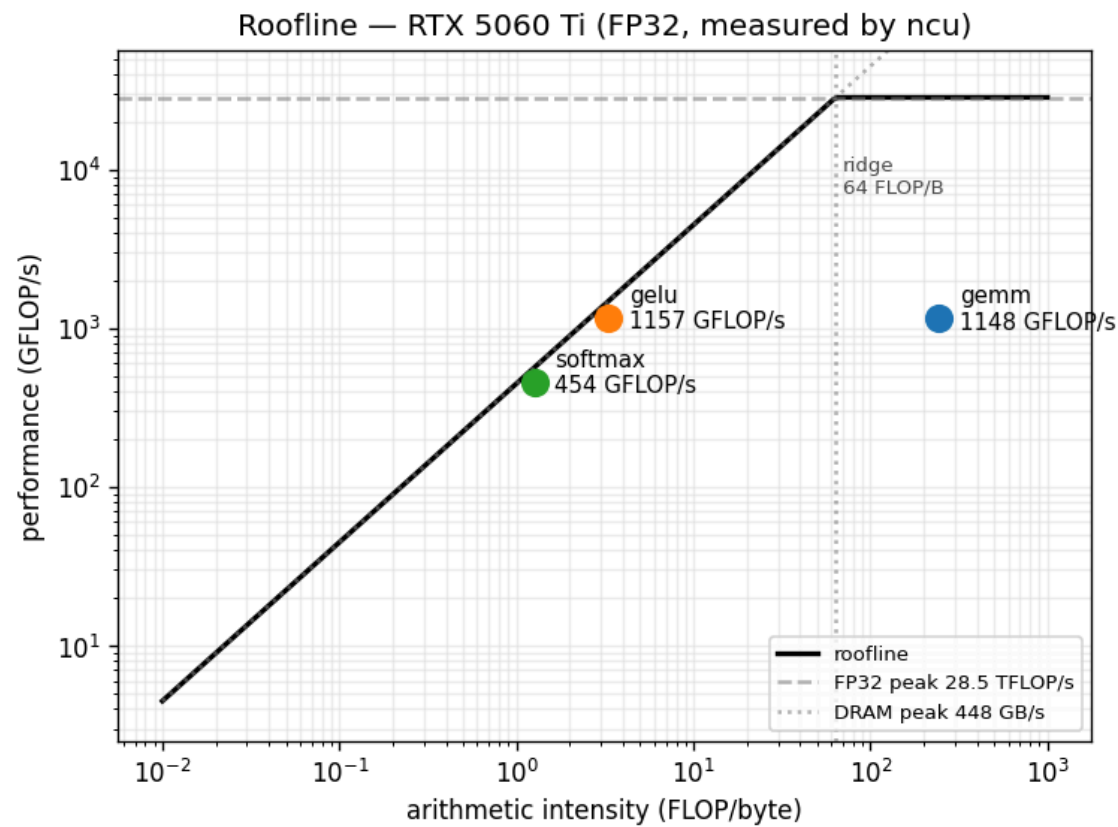
4.4 Block size sweep

Block size sweep (batch=16 S=128)



GEMM_TILE 8/16/32(재빌드, batch=16 S=128). latency 22.1/20.8/23.5ms. 작은 행렬(D=128)에서는 TILE=16이 최적(점유율/타일 오버헤드 균형).

4.5 Roofline

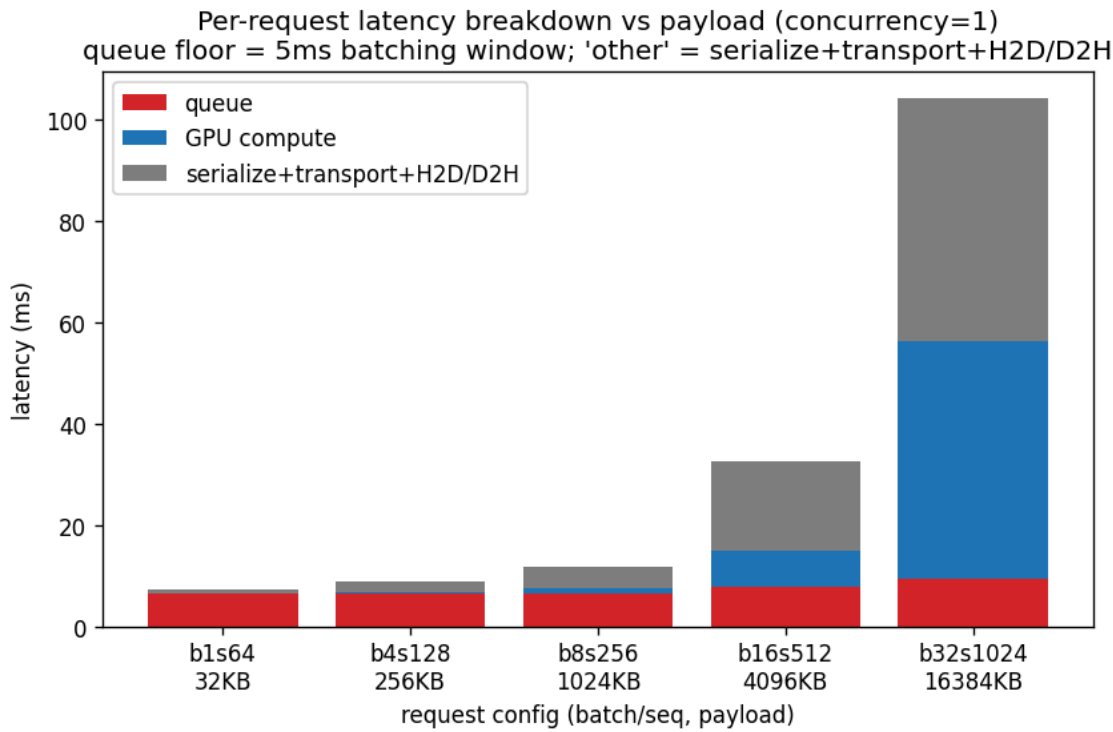


이 GPU의 실측 roofline. peak FP32≈28.5 TFLOP/s, DRAM≈448 GB/s, ridge=64 FLOP/B. ncu 실측 (FLOP=fadd+fmul+2·ffma, dram__bytes, gpu__time_duration):

kernel	arithmetic intensity	achieved	DRAM BW	위치
gemm	244 FLOP/B	1148 GFLOP/s	4.7 GB/s	compute-bound, peak의 ~4%
gelu	3.3 FLOP/B	1157 GFLOP/s	348 GB/s	memory-bound, peak BW ~78%
softmax	1.3 FLOP/B	454 GFLOP/s	356 GB/s	memory-bound, peak BW ~79%

gelu/softmax는 메모리 대각선 근처(효율적), gemm은 compute roof 아래 큰 여유 → 직접 짠 tiled GEMM의 최적화 여지 (register blocking/vectorization 미적용).

4.6 Bottleneck Analysis



페이로드 32KB→16MB 스위치(동시성 1, 경합 없는 순수 per-request 분해). end-to-end latency를 **queue / GPU compute / serialize+transport+H2D·D2H(other)** 로 분해(ms):

config	payload	queue	compute	other
b1 s64	32KB	6.5	0.1	0.7
b4 s128	256KB	6.5	0.3	2.0
b8 s256	1MB	6.5	1.2	4.1
b16 s512	4MB	7.7	7.2	17.6
b32 s1024	16MB	9.5	46.8	47.9

도출되는 주요 병목: - **Dispatcher 배칭 윈도우(고정 오버헤드)**: queue가 페이로드와 무관하게 ~6.5ms로 평탄 = dispatcher가 단일 요청도 항상 5ms 윈도우를 대기하기 때문. 작은 페이로드에서는 이 윈도우가 지배적 latency. → 유휴 워커가 있으면 즉시 flush하도록 개선 가능. - **Serialization/Communication overhead**: other는 페이로드에 비례(32KB 0.7ms → 16MB 48ms). protobuf encode/decode + 2-hop TCP + H2D/D2H 복사 합산. - **GPU bottleneck**: 큰 페이로드에서 compute 급증(16MB 46.8ms)하여 **window/transport-bound → compute-bound 크로스오버**. - **Queue contention(고동시성 시)**: 단일 워커를 mutex로 직렬화하므로 동시성 ↑ 시 이전 배치 대기로 queue가 추가 증가. 워커 스케일링이 sub-linear한 근본 원인(단일 GPU 공유, § 4.3).

검증 — queue/communication이 큰 것은 버그인가? (No) 동일 1MB 요청을 3가지 조건으로 통제 측정:

시나리오	latency	queue	compute	other(comm)
A. 워커 직접 호출, 동시성 1	2.1	0.0	1.2	0.9
B. dispatcher 경유, 동시성 1	11.9	6.5	1.2	4.3
C. dispatcher 경유, 동시성 16	80.0	20.8	17.2	42.0

- A에서 통신 오버헤드는 1MB당 0.9ms로 매우 작다 → 통신 자체엔 누수/버그 없음.

- B의 queue 6.5ms는 5ms 배칭 윈도우(고정), other 4.3ms는 2-hop(1MB×왕복) 추가 비용.
- C의 큰 값은 **배치가 16×1MB를 하나의 16MB 워커 호출로 병합한 결과** — 각 요청의 compute/other가 병합 배치 전체의 연산/통신을 반영(compute 17ms = batch 128의 forward, other 42ms = 16MB 직렬화/전송/복사).
queue+compute+other=latency로 내부 일관.

즉 큰 queue/communication은 **포화·고동시성·배치 상태의 정상적 결과**이며 측정 오류가 아니다. 위 breakdown 차트는 혼동을 피하기 위해 **동시성 1**로 측정해 고정 윈도우·순수 통신·연산을 분리해 보여준다.

참고: other는 latency - queue - compute 잔여값으로 직렬화/전송/H2D·D2H가 합산돼 있다. H2D/D2H를 별도 cudaEvent+proto 필드로 분리하는 4-way 정밀 분해는 향후 과제.

5. Reproducibility & Documentation

5.1 빌드

```
cmake -B build -S .          # -DCUDA_ARCH=native 로 fallback 가능
cmake --build build -j
ctest --test-dir build       # kernels / gen_fixtures / block (3/3 통과)
```

요구: CUDA 12.x, CMake≥3.24, C++17, OpenMP, gRPC/Protobuf(sudo apt install libgrpc++-dev protobuf-compiler-grpc libprotobuf-dev). gRPC 미설치 시 커널+테스트만 빌드(서빙 바이너리 skip).

5.2 실행

```
python3 tests/ref_block.py --out fixtures
WORKERS=$(bash scripts/launch_workers.sh 2 50061 fixtures build)
./build/dispatcher --port 50050 --workers "$WORKERS" --window-ms 5 --max-batch 8 &
./build/client --target localhost:50050 --fixtures fixtures          # 검증
./build/client --target localhost:50050 --requests 400 --concurrency 16 \
    --batch 8 --seq_len 256 --csv results/run.csv --tag demo # 부하
```

5.3 실험 + 그래프 (one-command)

```
bash scripts/run_experiments.sh # 스왑 → results/*.csv, ncu roofline, plot → results/*.png
```

5.4 산출물

- 소스: kernels/(커널), src/(worker/dispatcher/client), proto/infer.proto
- 빌드: CMakeLists.txt, docker/Dockerfile+docker-compose.yml
- 테스트: tests/, 스크립트: scripts/
- README.md(빌드/실행/실험 절차 + 결과 그래프 임베드), 결과 차트/CSV는 results/에 포함.

5.5 한계 및 향후 과제

- LayerNorm/residual/dropout 미포함(블록 단순화).
- GEMM이 peak의 ~4% → register blocking/벡터화로 개선 여지.

- 단일 GPU 공유로 워커 스케일링 한계 → 다중 GPU 또는 MPS/스트림 다중화.
- H2D/D2H 별도 계측으로 4-way 병목 분해 정밀화.

5.6 루브리크 대응 요약

항목(배점)	본 프로젝트
Architecture & Middleware (30)	2-hop gRPC 파이프라인(Client→Dispatcher→Worker), 명확한 구조 분리, 배칭/라우팅
Parallel Computing & Acceleration (30)	직접 작성 CUDA 커널 6종 + OpenMP gather/scatter (이종 가속)
Performance Evaluation & Analysis (25)	batch/seq/worker/block-size 스윙 + roofline + 병목 breakdown(정량)
Reproducibility & Documentation (15)	CMake/Docker, 원커맨드 실험 스크립트, README, ctest 자동 검증